
3 BOXES LUXURY

Functional & Technical Document

Version:	1.1
Platform:	Next.js 16 + Flutter (Android)
Database:	SQLite via Prisma ORM
Date:	May 09, 2026
Classification:	Internal / Confidential

Table of Contents

- 1. Executive Summary
- 2. System Architecture
- 3. Technology Stack
- 4. Database Schema
- 5. API Endpoints Reference
- 6. Frontend Components (Web)
- 7. Flutter Android App
- 8. AI Virtual Try-On Feature
- 9. Multi-Currency & i18n
- 10. Security & Authentication
- 11. API Test Results
- 12. Flutter App Test Results
- 13. Known Issues & Recommendations
- 14. Deployment Guide

1. Executive Summary

3 BOXES LUXURY is a full-stack curated luxury gifting e-commerce platform designed for the Indian market with global scalability. The platform features a Next.js 16 web portal, a Flutter-based Android app, AI-powered virtual try-on, multi-currency support with 31 currencies, 10-language i18n, corporate gifting portal, admin dashboard, and affiliate product integration from platforms like Myntra, Nykaa, CaratLane, Tanishq, BlueStone, and Voylla.

Key Business Capabilities:

- 57 luxury products across 11 categories (Jewelry, Watches, Sarees, Fashion, Fragrances, etc.)
- AI Virtual Try-On with 4 generation strategies and VLM face/product verification scoring
- Multi-currency support: 31 currencies with real-time conversion from INR base
- 10-language i18n: English, Hindi, Arabic, Japanese, Chinese, Korean, French, German, Spanish, Portuguese
- Corporate gifting with campaign management, recipient tracking, and custom branding
- Affiliate product integration from 8+ Indian luxury platforms
- Gift Builder for custom gift box creation and AI Gift Assistant for recommendations
- Admin dashboard with product, order, user, and inventory management
- Role-based access control: Admin, User, Agent, Team, Corporate roles
- Progressive Web App with Android APK distribution

2. System Architecture

The system follows a monolithic Next.js architecture with client-server separation:

Layer	Technology	Purpose
Frontend (Web)	React 19 + Next.js 16 App Router	SSR/CSR hybrid, single-page app experience
Frontend (Mobile)	Flutter 3.x + Dart	Native Android app with Provider state management
UI Framework	Tailwind CSS 4 + shadcn/ui (Radix)	Component library with 45+ UI components
State Management	Zustand (client) + TanStack Query (server)	Global state + async data fetching/caching
Backend API	Next.js Route Handlers (App Router)	RESTful API with 40+ endpoints
Database	SQLite via Prisma ORM	30+ models, 57 products, 11 categories
Authentication	Custom JWT-like token auth + bcryptjs	Registration, login, 2FA, session management
AI Services	ZAI SDK (VLM + Image Gen) + Sharp	Virtual try-on, gift recommendations
Reverse Proxy	Caddy	Gateway on port 81 forwarding to Next.js :3000
PWA	Service Worker + Manifest	Installable web app with offline support
Payments	Razorpay / Stripe (API ready)	Payment sessions, verification webhook

3. Technology Stack

3.1 Web Platform Dependencies

Package	Version	Purpose
next	^16.1.1	Full-stack React framework with App Router
react / react-dom	^19.0.0	UI library
prisma	^6.11.1	Type-safe ORM for SQLite
z-ai-web-dev-sdk	^0.0.17	AI services (VLM, image generation)
sharp	^0.34.5	Server-side image processing
zustand	^5.0.6	Lightweight client state management
@tanstack/react-query	^5.82.0	Server state management & caching
framer-motion	^12.23.2	Animation library
lucide-react	^0.525.0	Icon library (800+ icons)
bcryptjs	^3.0.3	Password hashing
jsonwebtoken	^9.0.3	JWT token generation
zod	^4.0.2	Schema validation
recharts	^2.15.4	Data visualization charts
@radix-ui/*	various	45+ accessible UI primitives
tailwindcss	^4	Utility-first CSS framework

3.2 Flutter Android App Dependencies

Package	Version	Purpose
provider	^6.1.5+1	State management (ChangeNotifier)
http	^1.6.0	HTTP client for API communication
shared_preferences	^2.5.5	Local key-value storage
cached_network_image	^3.4.1	Cached image loading with placeholders
google_fonts	^8.1.0	Google Fonts integration
flutter_svg	^2.2.4	SVG rendering
shimmer	^3.0.0	Loading shimmer effects
go_router	^17.2.3	Declarative routing
intl	^0.20.2	Internationalization & number formatting
motion_tab_bar	^2.0.4	Animated bottom navigation
badges	^3.2.0	Notification badges
animations	^2.2.0	Container transitions & fade effects

4. Database Schema

The database uses SQLite via Prisma ORM with 30+ models. Below is the complete schema overview:

4.1 Core Models

Model	Key Fields	Relationships
Category	id, name, slug (unique), description, image	has many Products
Product	id, productNumber (unique), name, slug, price, compareAtPrice, images (JSON), category, featured, tags (JSON), reasons (JSON), reviews (JSON), createdAt, updatedAt	belongs to Category, has many ProductVariants, has many ProductImages, has many Invoices
ProductVariant	id, name, sku, price, stock, attributes (JSON), image	belongs to Product
ProductImage	id, url, alt, sort, isActive	belongs to Product
Vendor	id, name, slug, contactName, email, gstNumber	has many Products, Invoices

4.2 User & Authentication Models

Model	Key Fields	Purpose
User	id, email (unique), name, password (hashed), role (admin/user/agent/vendor/buyer), twoFactorEnabled, appToken, createdAt, updatedAt	Central user identity and role management
Session	id, token (unique), userId, ipAddress, userAgent, expiresAt	Active session management
UserPermission	id, userId, permission (unique pair)	Granular permission system (e.g., 'products.manage')
AuditLog	id, userId, action, entity, entityId, details (JSON)	Comprehensive audit trail

4.3 Order & Payment Models

Model	Key Fields	Purpose
Order	id, orderNumber, email, address, subtotal, shipping, tax, discount, total, status (5 stages), paymentStatus, deliveryType, giftWrapping	Full order lifecycle management
OrderItem	id, orderId, productId, name, price, quantity, giftWrapping	Order line items
PaymentSession	id, orderId, provider (razorpay/stripe), amount, status, paymentId	Payment gateway sessions
OrderInvoice	id, orderId (unique), invoiceNumber (unique), amount, tax, total, pdfUrl	Invoice generation
OrderTrackingEvent	id, orderId, status, description, location	Shipment tracking events

4.4 Corporate Gifting Models

Model	Key Fields	Purpose
CorporateAccount	id, companyName, slug, industry, gstNumber, contactName, creditLimit, discountPercentage, approvalStatus	Corporate client onboarding
CorporateMember	id, corporateId, email, role (3 types), status	Team member management
CorporateBranding	id, corporateId (unique), logoUrl, primaryColor, packagingType, shippingPolicy	Customize corporate branding
CorporateCampaign	id, corporateId, name, occasion, budgetPerRecipient, status (3 stages), createdAt	Multi-stage gifting campaigns
CampaignRecipient	id, campaignId, name, email, designation, giftStatus, orderId	Individual gift recipients

4.5 Platform Integration Models

Model	Key Fields	Purpose
PlatformIntegration	id, name, slug, baseUrl, autoSync, syncInterval, affiliateTag, commissionRate	External platform connectors (Myntra, Nykaa, etc.)
SyncLog	id, integrationId, type, status, productsFound, productsAdded	Product sync audit trail
PartnerCategoryMap	id, integrationId, partnerCatName, localCatId	Category mapping between platforms
AffiliateClick	id, productId, platform, sourceUrl, referralCode, ipAddress	Affiliate click tracking

4.6 Supporting Models

Model	Key Fields	Purpose
Cart / CartItem	sessionId, productId, quantity, giftWrapping, greetingMessage, shipping	Shopping cart with gift options
WishlistItem	userId, productId (unique pair)	Product wishlist
Review	productId, orderId, userId, userName, rating (1-5), title, comment	Product reviews
Offer	code (unique), type (percentage/fixed), value, minOrder, maxDiscount, validFrom, validTo, usageLimit	Promotional offers
CurrencyRate	code (unique), name, symbol, rate (against INR)	Multi-currency exchange rates
GeoCountry	code (unique), name, countryCode, languageCode, flagEmoji	Geo-location based defaults
InventoryLog	productId, type (in/out/adjustment/return), quantity, note	Stock movement audit
AccountEntry	entryNumber, type (debit/credit), category, amount, reference	Financial accounting
SupportTicket / Message	subject, status, priority, messages	Customer support system

5. API Endpoints Reference

The platform exposes 40+ RESTful API endpoints via Next.js Route Handlers:

5.1 Product & Catalog APIs

Endpoint	Method	Auth	Description
/api/products	GET	Public	List products with pagination, category/sort/source/occasion/recipient/price filters
/api/products/[id]	GET	Public	Get single product with full details
/api/categories	GET	Public	List all categories with product counts
/api/products/fix-images	POST	Admin	Fix product image URLs (bulk)

5.2 Authentication APIs

Endpoint	Method	Auth	Description
/api/auth/register	POST	Public	Register new user (email, password, name)
/api/auth/login	POST	Public	Login and receive auth token
/api/auth/me	GET	Token	Get current user profile
/api/auth/social	POST	Public	Social login (Google/Facebook/LinkedIn)
/api/auth/2fa/setup	POST	Token	Setup 2FA (TOTP)
/api/auth/2fa/verify	POST	Token	Verify 2FA code
/api/auth/verify-phone	POST	Token	Verify phone number via OTP
/api/auth/approve	POST	Admin	Approve/reject user registration

5.3 Shopping & Checkout APIs

Endpoint	Method	Auth	Description
/api/cart	GET/POST/DELETE	Session	Cart CRUD (requires x-session-id header)
/api/checkout	POST	Token	Place order with shipping, gift wrap, promo
/api/checkout/estimate	POST	Public	Estimate shipping cost
/api/offers	GET	Public	List active promotional offers
/api/offers/validate	POST	Public	Validate offer code and calculate discount
/api/wishlist	GET/POST/DELETE	Token	Wishlist management

5.4 Order & Payment APIs

Endpoint	Method	Auth	Description
/api/orders	GET	Token	List user orders (requires email param)
/api/orders/[id]	GET	Token	Get order detail

Endpoint	Method	Auth	Description
/api/orders/{id}/tracking	GET	Token	Get order tracking events
/api/orders/{id}/invoice	GET	Token	Get order invoice
/api/orders/{id}/refund	POST	Token	Request order refund
/api/payments/create-session	POST	Token	Create Razorpay/Stripe payment session
/api/payments/verify	POST	Public	Verify payment webhook callback

5.5 AI & Intelligence APIs

Endpoint	Method	Auth	Description
/api/try-on	POST	Public	Start AI virtual try-on job (returns jobId)
/api/try-on?jobId=xxx	GET	Public	Poll try-on job status/result
/api/gift-recommend	POST	Public	AI-powered gift recommendations
/api/image-proxy	GET	Public	Proxy external images (CORS, platform logos)

5.6 Admin APIs

Endpoint	Method	Auth	Description
/api/admin/products	GET/POST	Admin	Product CRUD (list/create)
/api/admin/products/{id}	GET/PUT/DELETE	Admin	Product CRUD (read/update/delete)
/api/admin/categories	GET/POST	Admin	Category management
/api/admin/users	GET	Admin	List all users
/api/admin/users/{id}	PUT/DELETE	Admin	User management
/api/admin/permissions	GET/POST	Admin	Permission management
/api/admin/orders	GET	Admin	All orders (admin view)
/api/admin/corporate	GET	Admin	Corporate account management
/api/admin/corporate/{id}	PUT	Admin	Corporate account approval
/api/admin/campaigns	GET	Admin	Campaign management

5.7 Corporate Gifting APIs

Endpoint	Method	Auth	Description
/api/corporate/register	POST	Public	Register corporate account
/api/corporate/login	POST	Public	Corporate login
/api/corporate/profile	GET/PUT	Token	Corporate profile management
/api/corporate/members	GET/POST	Token	Team member management
/api/corporate/branding	GET/PUT	Token	Custom branding settings
/api/corporate/campaigns	GET/POST	Token	Campaign management

Endpoint	Method	Auth	Description
/api/corporate/campaigns/[id]	GET/PUT	Token	Campaign detail/update
/api/corporate/campaigns/[id]/recipients	GET/POST	Token	Recipient management
/api/corporate/campaigns/[id]/submit	POST	Token	Submit campaign for processing

5.8 Platform & Utility APIs

Endpoint	Method	Auth	Description
/api/geo	GET	Public	Geo-detect country, currency, language
/api/currency/rates	GET	Public	Get 31 currency exchange rates
/api/vendors	GET/POST	Admin	Vendor management
/api/vendors/[id]	GET/PUT/DELETE	Admin	Vendor CRUD
/api/partners	GET/POST	Admin	Platform integration management
/api/partners/[id]/sync	POST	Admin	Trigger product sync from partner
/api/partners/[id]/category-maps	GET/POST	Admin	Category mapping management
/api/inventory	GET	Admin	Inventory status overview
/api/inventory/[productId]	GET/POST	Admin	Product inventory logs
/api/accounting	GET/POST	Admin	Accounting entries
/api/invoices	GET/POST	Admin	Invoice management
/api/invoices/[id]	GET	Admin	Invoice detail
/api/reviews	GET/POST	Public/Token	Product reviews
/api/support/tickets	GET/POST	Token	Support ticket management
/api/support/tickets/[id]/messages	GET/POST	Token	Ticket messages
/api/affiliate/click	POST	Public	Track affiliate click
/api/affiliate/stats	GET	Token	Affiliate click statistics

6. Frontend Components (Web)

The web frontend is built as a single-page application using React 19 with Next.js 16 App Router. All views are rendered client-side via Zustand state-driven view switching. The application uses 45+ shadcn/ui components built on Radix UI primitives.

Component	File	Purpose
Header	header.tsx	Sticky navigation with search, locale switcher, cart, auth, gift builder
HeroSection	hero-section.tsx	Animated hero with bokeh lights, golden rays, particles
CategoryGrid	category-grid.tsx	Category cards with images and product counts
ProductGrid	product-grid.tsx	Filterable product grid with sort, source, occasion, recipient, price filters
ProductCard	product-card.tsx	Product card with shimmer loading, platform badges, affiliate tracking
ProductDetail	product-detail.tsx	Full detail view with gallery, try-on, reviews, wishlist, related products
TryOnDialog	product-detail.tsx	AI virtual try-on wizard (upload, preview, generating, result steps)
CartView	cart-view.tsx	Cart with item management, promo codes, order summary
CheckoutView	checkout-view.tsx	Multi-step checkout with shipping, gift options, payment
OrderConfirmation	order-confirmation.tsx	Order success page with tracking info
OrderHistory	order-history.tsx	Past orders with status filters and invoice download
AuthDialog	auth-dialog.tsx	Login/register with social auth, 2FA support
AdminDashboard	admin-dashboard.tsx	Product, order, user, inventory, vendor management
UserDashboard	user-dashboard.tsx	User profile, order stats, settings
CorporateDashboard	corporate-dashboard.tsx	Corporate gifting campaigns, branding, member management
GiftAssistant	gift-assistant.tsx	AI-powered gift recommendation chatbot
GiftBuilder	gift-builder.tsx	Custom gift box builder with box sizes, product picker, message card
LocaleSwitcher	locale-switcher.tsx	Currency and language selector popover
Footer	footer.tsx	Site footer with links and app download
AppDownloadSection	app-download-section.tsx	Android app download CTA section
AppDownloadBanner	app-download-banner.tsx	Floating app download banner

7. Flutter Android App

The Android app is built with Flutter 3.x and follows a Provider-based state management pattern. It features a dark luxury theme matching the web portal with gold accents.

7.1 Screen Inventory

Screen	File	Features
HomeScreen	home/home_screen.dart	Hero banner, featured products, categories, all products, pull-to-refresh
CategoryScreen	home/category_screen.dart	Category browsing with product listing by category
ProductDetailScreen	product/product_detail_screen.dart	Image carousel, pricing, stock, variants, related products
CartScreen	cart/cart_screen.dart	Item list, quantity controls, swipe-to-delete, order summary, promo
CheckoutScreen	checkout/checkout_screen.dart	Shipping form, promo code, payment selection, order placement
AuthScreen	auth/auth_screen.dart	Login/register forms, social auth buttons, password toggle
OrdersScreen	orders/orders_screen.dart	Order history with status tabs, order detail, invoice download
WishlistScreen	wishlist/wishlist_screen.dart	Wishlist with swipe-to-remove, move-to-cart
GiftAssistantScreen	gift/gift_assistant_screen.dart	Step-by-step AI gift recommendation wizard
GiftBuilderScreen	gift/gift_builder_screen.dart	Custom gift box builder with box size, products, message
AdminDashboardScreen	admin/admin_dashboard_screen.dart	Product, order, user, category management
UserDashboard	dashboard/user_dashboard.dart	Profile, order stats, quick actions, settings
CorporateDashboard	dashboard/corporate_dashboard_screen.dart	Campaign management, branding, members

7.2 App Configuration

The app config at `lib/config/app_config.dart` defines API endpoints, colors, and defaults. **CRITICAL ISSUE:** `baseUrl` is currently empty string, which works for web deployment but will fail on native Android. Must be set to the actual server URL (e.g., `http://10.0.2.2:3000` for emulator).

8. AI Virtual Try-On Feature

The AI Virtual Try-On is a flagship feature allowing users to upload a selfie and see how luxury products look on them. It uses a multi-strategy pipeline with VLM verification scoring.

8.1 Pipeline Architecture

Step	Process	Details
1	Image Upload	User selfie uploaded, compressed to max 1536px, quality 0.92 JPEG
2	Job Creation	POST /api/try-on returns jobId, background processing starts
3	VLM Analysis (Parallel)	Person description + Product description extracted simultaneously
4a	Strategy A: edit-both	Edit with BOTH images (selfie + product) - best combined accuracy
4b	Strategy B: edit-selfie	Edit selfie only with text prompt - best face preservation
4c	Strategy C: edit-product	Edit product image with person description - best product accuracy
4d	Strategy D: create-detailed	Generate from descriptions only - fallback strategy
5	VLM Verification	Each result scored: Face Match (1-10) + Product Match (1-10)
6	Best Selection	Weighted score: 60% face + 40% product, early exit if face>=8 & product>=7
7	Result Delivery	Polling GET /api/try-on?jobId=xxx returns image + scores + suggestions

8.2 Category-Specific Prompting

The system uses category-aware prompts for accurate product placement: Sarees use '768x1344' portrait images with 'draped in traditional Indian style with pallu over shoulder'; Jewelry differentiates earrings, necklaces, bracelets, rings by name; Watches use '864x1152' with 'wearing the watch on the wrist'. Product descriptions from VLM include exact color names, material textures, and pattern details.

9. Multi-Currency & Internationalization

9.1 Supported Currencies (31)

All product prices are stored in INR (base currency). The CurrencyRate model stores live exchange rates. The useCurrency() hook provides format() and convert() functions with locale-specific formatting (e.g., \$1,234.56 for USD, 1.234,56 EUR for German). The /api/currency/rates endpoint returns all 31 rates. Auto-detection via /api/geo sets currency based on user's country.

9.2 Supported Languages (10)

Language	Code	Translation File
English	en	en.json (complete)
Hindi	hi	hi.json
Arabic	ar	ar.json (RTL support)
Japanese	ja	ja.json
Chinese	zh	zh.json
Korean	ko	ko.json
French	fr	fr.json
German	de	de.json
Spanish	es	es.json
Portuguese	pt	pt.json

10. Security & Authentication

Password Security: bcryptjs with salt rounds for hashing. Passwords never stored in plain text.

Token-Based Auth: Custom UUID token system stored in Session model. Tokens expire and track IP/user agent.

Two-Factor Auth: TOTP-based 2FA setup and verification. Admin-enforced 2FA requirement supported.

Role-Based Access: 5 roles: admin, user, agent, team, corporate. Admin has granular permissions (UserPermission model).

CORS Headers: Global CORS headers configured in next.config.ts allowing all origins for API access.

User Approval: New registrations require admin approval (approvalStatus: pending/approved/rejected/suspended).

Audit Logging: All sensitive actions (login, role changes, approval changes) logged in AuditLog model.

Image Proxy: External images proxied through /api/image-proxy to avoid CORS issues and track usage.

11. API Test Results

All 25 API endpoint groups were tested against the running server. Results:

Endpoint	Status	Result
/api/categories	200	11 categories loaded correctly
/api/products?limit=5	200	57 products, pagination working
/api/products/[id]	200	Full product detail with category
/api/offers	200	2 offers: WELCOME10 (10%), FLAT25
/api/offers/validate	200	Validation works; needs 'subtotal' field
/api/inventory	401/403	Admin-only - correctly protected
/api/auth/register	200	User registration works
/api/auth/login	200	Login returns auth token
/api/reviews?productId=X	200	Returns reviews (empty for new products)
/api/reviews POST	201	Review creation works
/api/wishlist	401/200	Requires auth; works with token
/api/cart	200	Requires x-session-id header
/api/geo	200	Returns IN/INR/hi with flag emoji
/api/currency/rates	200	31 currency rates returned
/api/vendors	401/403	Admin-only - correctly protected
/api/accounting	401/403	Admin-only - correctly protected
/api/admin/categories	200	WARNING: accessible to non-admin users
/api/admin/users	403	Admin-only - correctly protected
/api/admin/products	403	Admin-only - correctly protected
/api/corporate/register	405/400	POST-only; validates required fields
/api/gift-recommend	200	AI recommendations with catalog picks
/api/support/tickets	403	Admin-only - correctly protected
/api/orders	200	Requires auth + email query param
/api/invoices	403	Admin-only - correctly protected
/api/partners	401/403	Admin-only - correctly protected

11.1 Security Gap Found

ISSUE: /api/admin/categories returns full data (HTTP 200) even for non-admin regular users. All other /api/admin/* endpoints properly enforce admin-only access. This is a security inconsistency that should be fixed by adding admin role check to the categories admin endpoint.

12. Flutter App Test Results

12.1 Critical Issues Found

#	Issue	Impact	Priority
1	AppConfig.baseUrl is empty string	All API calls fail on native Android - relative URLs resolve to http://... which is invalid	CRITICAL
2	Auth token never set after login/register	AppProvider.login() never calls _api.setAuthToken(). All authenticated endpoints return 401	CRITICAL
3	Auth token not persisted across app restarts	shared_preferences dependency exists but is never used. User is logged out on every restart	CRITICAL
4	Cart/Wishlist/Orders never loaded from backend	ApiService methods exist but AppProvider never calls getCategory(), getWishlist(), getOrders()	CRITICAL
5	Checkout is mock - never calls backend	_placeOrder() just does Future.delayed(2s). ApiService.checkout() exists but is never used	CRITICAL
6	Promo code bug in checkout	provider.formatPrice(500) may crash due to incorrect context access	High

12.2 High Priority Issues

#	Issue	Location
1	Category image not using AppConfig.getImageUrl()	category_screen.dart line 419
2	Wishlist only shows products already in loaded list	wishlist_screen.dart
3	Orders screen uses mock data only	orders_screen.dart - _generateMockOrders()
4	User dashboard stats are hardcoded	user_dashboard.dart
5	No email format validation on registration	auth_screen.dart
6	Admin/Corporate API paths hardcoded in ApiService	api_service.dart lines 187-238
7	No network timeout on API calls	api_service.dart
8	Missing url_launcher dependency for external links	pubspec.yaml

12.3 Working Features

- UI/UX Design: Dark luxury theme with gold accents is cohesive and polished
- Loading States: Most screens properly handle loading with spinners/shimmer
- Empty States: Thoughtful empty state messaging across all screens
- Pull-to-Refresh: Implemented on home and orders screens
- Image Handling: CachedNetworkImage with shimmer placeholders and error fallbacks
- Role-Based Access: Admin and Corporate dashboards properly check user roles
- Navigation: IndexedStack preserves tab state; named routes defined
- Admin Panel: Full CRUD capability with proper API integration

Overall Assessment: The Flutter app has a polished, production-quality UI layer, but the backend integration is approximately 30-40% complete. The critical path from login -> authenticated API calls -> cart sync -> checkout -> order placement is non-functional due to missing token management and empty baseUrl.

13. Known Issues & Recommendations

13.1 Web Portal Issues

#	Issue	Recommendation
1	/api/admin/categories accessible to non-admin users	Add admin role check to admin categories endpoint
2	Cart requires custom x-session-id header (non-standard)	Document in API docs or switch to cookie-based sessions
3	Gift recommend requires POST with email field	Document required fields clearly; consider making email optional
4	Offers validate requires subtotal alongside code	Document or make subtotal optional with default 0
5	Orders endpoint requires email query param even with auth token	Use auth token to determine user, make email optional
6	HMR module identity can break on filename case changes	Use consistent kebab-case for all component filenames

13.2 Flutter App Recommended Fixes (Priority Order)

#	Fix	Effort
1	Set AppConfig.baseUrl to actual backend URL for Android	Low
2	Extract and set auth token after login/register in AppProvider	Medium
3	Persist auth token using shared_preferences	Medium
4	Load cart/wishlist/orders from backend in AppProvider.initialize()	Medium
5	Connect checkout to backend via ApiService.checkout()	Medium
6	Fix promo code bug in checkout screen	Low
7	Use AppConfig.getImageUrl() consistently in category screen	Low
8	Move admin/corporate API paths to AppConfig constants	Low
9	Add url_launcher to pubspec and implement external product links	Low
10	Extract shared utility for _getPlatformColor and shipping config	Low

14. Deployment Guide

14.1 Web Portal Deployment

- 1. Install dependencies: `bun install`
- 2. Configure environment: Set DATABASE_URL, JWT_SECRET in .env
- 3. Push database schema: `bun run db:push`
- 4. Seed initial data: `bun run prisma/seed.ts`
- 5. Start dev server: `bun run dev` (port 3000)
- 6. Production build: `bun run build` (outputs to .next/standalone/)
- 7. Start production: `NODE_ENV=production node .next/standalone/server.js`
- 8. Configure reverse proxy (Caddy/Nginx) to forward to port 3000

14.2 Android App Deployment

- 1. Set AppConfig.baseUrl to production server URL
- 2. Fix auth token management (see Section 13.2)
- 3. Run `flutter pub get`
- 4. Build APK: `flutter build apk --release`
- 5. Output: `build/app/outputs/flutter-apk/app-release.apk`
- 6. For Play Store: `flutter build appbundle --release`

14.3 Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	SQLite connection string (file:./db/custom.db)
JWT_SECRET	Yes	Secret key for token signing
NEXT_PUBLIC_BASE_URL	No	Base URL for metadata (defaults to localhost:3000)